Native hyperelliptic index calculus, genus 2–4: engine (S1) and solver (S2a)

Study: `research/hec_ic_speedup_20261113/` (PLAN.md, S1-RESULTS.md, S2-RESULTS.md).
Record: `benchmarks/records/20261114T0356Z-hec-native-ic.json`, run at commit `93047a9f60` (branch `research/hec-native-ic-20261113`).
Command: `run_record.sh rec-20261113` then `summarise.sh rec-20261113`; raw output in `rec-20261113/` (`biguint.txt`, `native_both.txt`, `binaries.sha256`, `host.txt`, `summary.jsonl`).
Host: one arm64 macOS laptop (Darwin 27.2.0), single thread, isolation L0 (development host, nothing else scheduled, no pinning; CPU model not recordable from the sandbox).
Class: engineering on the native engine. Same algorithm, same counted work; wall clock only. No `S` claim, no advance, not validated.

What was built and measured

- **S1, a native engine** (`crates/hec-native-ic`, no dependencies): Montgomery `u64` arithmetic in $\mathbb{F}_p$, fixed-array polynomials, Cantor composition with a degree-1 fast path, the round-six smoothness oracle (parity filter, then $\gcd(u, x^p - x)$, then equal-degree splitting), the round-eight factor-base walk (64 places, `d_step_in = 4`) and Markowitz elimination mod ℓ. It runs the same rows as the frozen BigUint engine (`examples/hyperelliptic_ic_large_prime.rs`, round eight).
- **S2a, a faster solver.** The S1 solver pushed a row id onto the holder list of every column of every updated row, so the lists of the about 64 hot step columns grew by one per row operation. `Solver::Fast` pushes only for columns new to the row, keeps no list for k and reuses one merge buffer. Pivot choice is the same code.

Curves: $y^2 = f(x)$ over $\mathbb{F}_p$ with $f = x^5 + 3x^3 + 2x^2 + x + c$ (genus 2), $x^7 + x^3 + cx + 1$ (genus 3) and $x^9 + x^3 + cx + 1$ (genus 4); c, the subgroup order ℓ and the base points come from `instance::pick_instance` and equal round eight's on every row. ICV1 slugs name elliptic curves only, so these Jacobians are named by equation and ℓ.

Pre-registered verdicts

id	expectation (PLAN.md)	verdict
G1–G4	group law, oracle vs brute force, identical instances, every k verified (S1)	hold
E1	native smoothness within a band of BigUint's (S1)	falsified as registered (bad band); like-for-like yields agree
E2	native engine ≥ 10× faster on every row (S1)	**falsified**: 4–18×; the native solve was 55–82 % of genus-2 time
E3	`Fast` and `Reference` agree exactly on k, pivot-sequence hash, pivots, row ops, mulmods on all 55 runs (S2)	**holds**, in three runs
E4	genus 2, $p \geq 503$: mean solve time falls ≥ 2× (S2)	**holds**: 2.9–4.3× (runs 1–2), 3.2–3.4× (this run)

The recorded run

Means over seeds 20260916–20260920 (five per row and arm). BigUint ms is that engine's own `wall_ic_ms` at $\theta = 1$. "Native S1" and "native S2a" are the same binary with `Solver::Reference` and `Solver::Fast`, alternating order by seed. All times exclude instance selection. Ratios and shares are derived from the means (`summarise.sh`); the solve ratio is also shown on medians.

g	p	ℓ	m	BigUint ms	native S1 ms	native S2a ms	BigUint / S2a	solve × mean / median	S2a solve share
2	101	4663	46	2.81	0.33	0.22	13×	2.31 / 2.29	40 %
2	251	61667	123	7.32	1.19	0.61	12×	2.68 / 2.79	56 %
2	503	65179	259	13.53	3.15	1.37	9.8×	3.19 / 3.05	59 %
2	1009	241259	478	25.66	7.32	3.38	7.6×	3.43 / 3.45	47 %
2	2003	2001787	1000	50.39	13.08	5.50	9.2×	3.40 / 3.40	57 %
2	4001	16050337	2005	101.32	26.26	11.61	8.7×	3.44 / 3.44	52 %
3	61	124459	33	8.00	0.56	0.43	18.6×	2.69 / 2.67	18 %
3	211	4620611	104	22.66	2.95	1.67	13.6×	3.62 / 3.56	28 %
3	401	31525763	196	41.33	5.64	3.27	12.6×	3.89 / 3.95	25 %
4	31	239753	16	19.24	1.07	1.02	18.9×	2.58 / 2.61	3 %
4	61	16790591	36	21.91	1.61	1.31	16.7×	3.20 / 3.48	10 %

Checks in this run: 55/55 `GATE … identical` lines (E3), 110/110 native solves verify $k \cdot D_1 = D_2$, and all 55 BigUint (row, θ) lines report `ic_correct`. `solve_mulmods` per row equals the S1 run's.

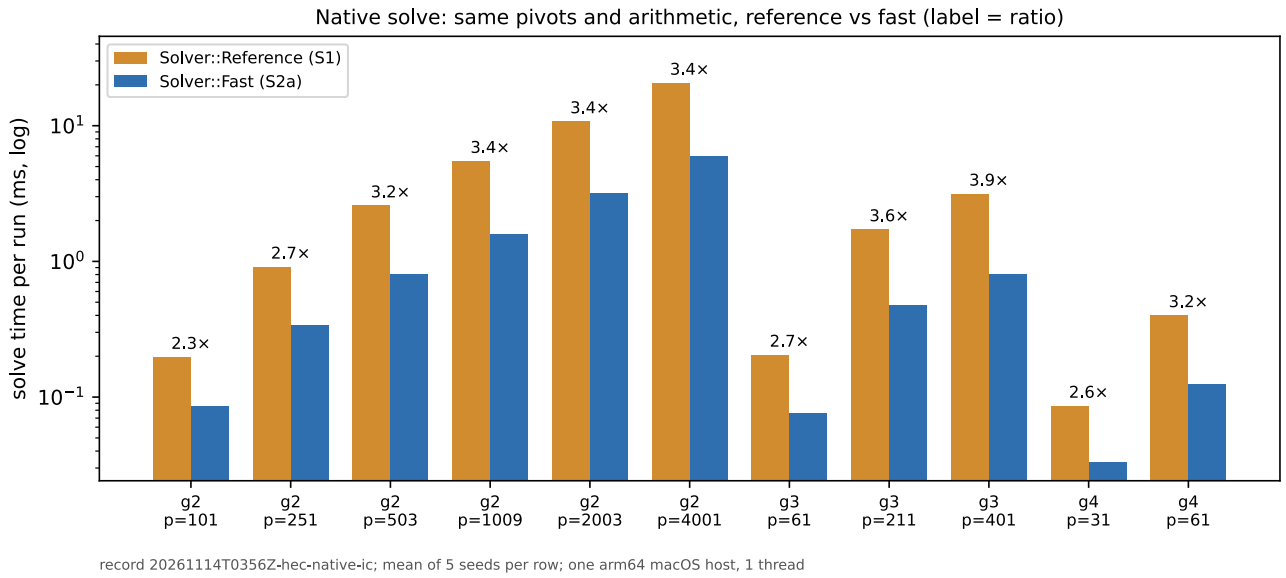


Figure 1: Native solve time per run, S1 solver vs S2a solver, same binary and rows. Label: mean ratio. Record 20261114T0356Z-hec-native-ic.

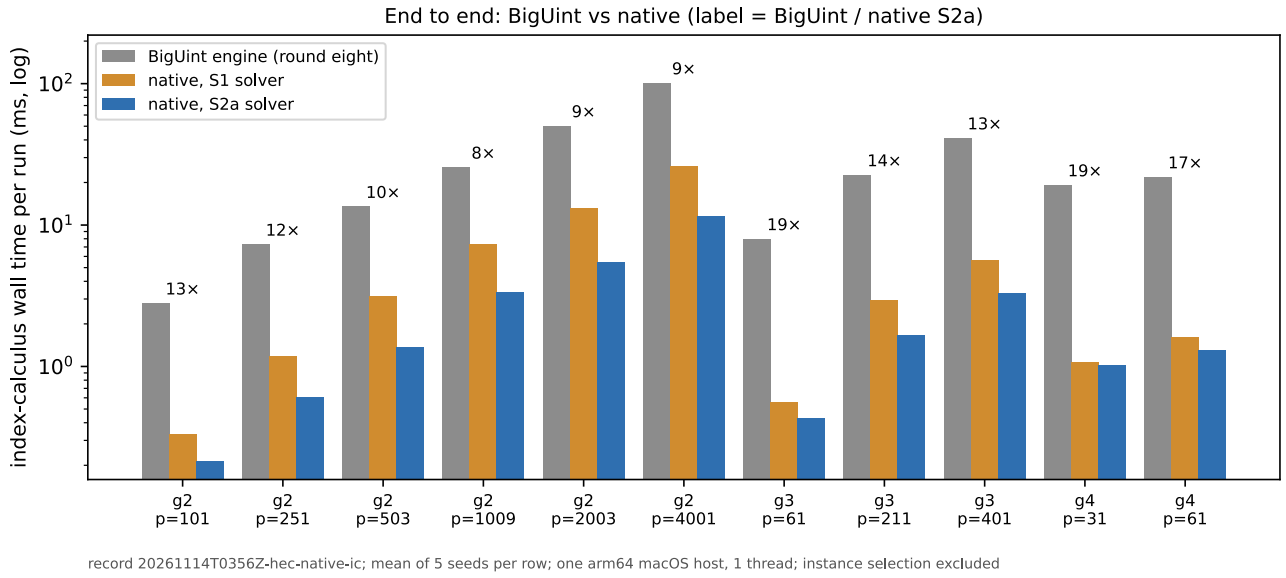


Figure 2: Index-calculus wall time per run: BigUint engine, native with the S1 solver, native with the S2a solver. Label: BigUint / native S2a. Same record.

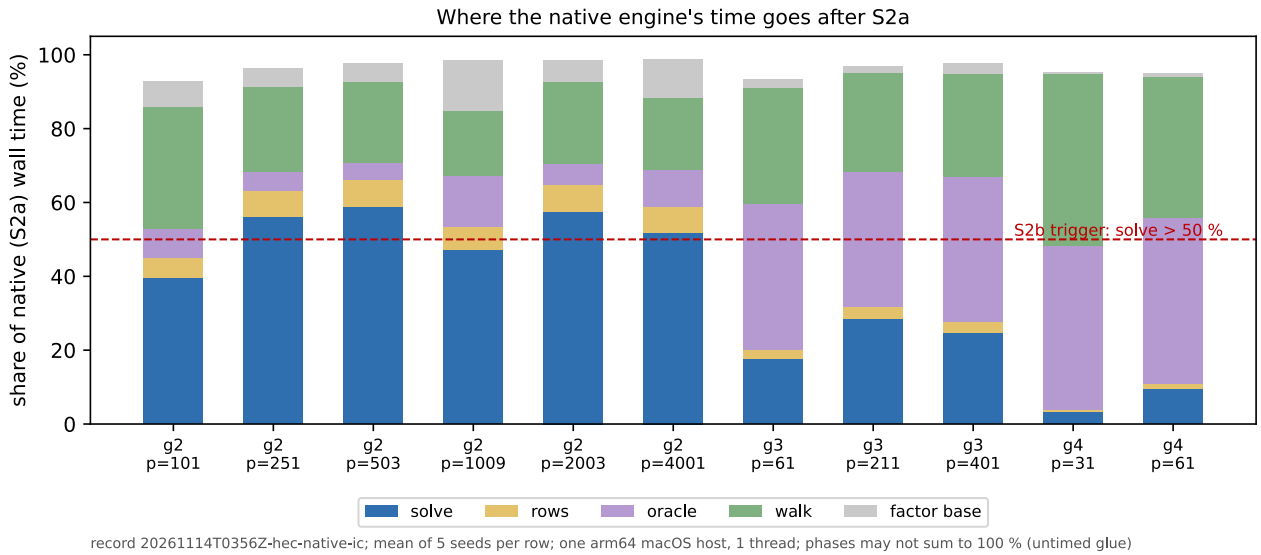


Figure 3: Share of native (S2a) wall time by phase. Solve is drawn at the bottom so it reads against the 50 % line that triggers S2b. Same record.

Reading

- The S2a change alone makes the native solve 2.3–3.9× faster on every row, with the counted work identical. End to end, the native engine with S2a is 7.6–13× faster than the BigUint engine at genus 2, 12.6–18.6× at genus 3 and 16.7–18.9× at genus 4 in this run. This end-to-end figure is post hoc: E2 was registered for S1 and stays falsified.
- **The solve is still the largest phase at genus 2.** The S2a solve share is 47–59 % for $p \geq 503$ in this run (53–64 % in runs 1–2). The trigger for S2b (a dense step-column block) fires cleanly at $p = 503$ only; elsewhere it is within run-to-run noise of 50 %.
- **At genus 3 and 4 the solve is no longer the lever.** The oracle and the walk take 64–91 % of native time there, so per-trial cost (plan track T2) comes before more solver work at those genera.

Earlier runs and their spread

S1 (one run of each engine, `s1/`) and S2 (two `HEC_SOLVER=both` runs, `s2/`) are reported in S1-RESULTS.md and S2-RESULTS.md. Timing noise between processes is large: on the same binary, the genus-2 $p = 2003$ reference solve took 22.8 ms in S2 run 1, 12.9 ms in run 2 and 10.8 ms here. Solve ratios are stable (2.9–4.3× at genus 2, $p \geq 503$, across three runs) because both arms run in the same process on the same rows.

External comparison (AGENTS.md §15): not run

No open implementation of hyperelliptic index calculus is known to the study. The double-large-prime implementations of Gaudry–Thomé–Thériault–Diem (Math. Comp. 2007) and Nagao (2007) are published, not reproduced, and their timings have not been extracted (PLAN.md §2 evidence gaps). Missing, and planned: the published 2LP timings (track T1), and the sparse solve against FLINT and CADO-NFS `bwc` on exported matrices (track T3).

Limits

One development host at isolation L0, no CPU model recorded, five seeds per row, wall clock only. Instance selection (BSGS order) is excluded on both engines. The rows are round eight's small instances ($p \leq 4001$ at genus 2); nothing here says how either engine scales beyond them. The record is not validated: no `VALIDATION.json`, replay certificate or matched rho reference, so the `benchmarks/` entry stays out of the admitted table.